

Jhonny Retail Agent POC Plan

STATUS

Review-ready local POC plan

AUDIENCE

Tiago, Rodrigo, founding team, and pilot reviewers

PREPARED

May 10, 2026

Table of Contents

1. Product Goal
2. Current Implementation
3. Target Architecture
4. Data Architecture
5. App Experience
6. Agent And Tooling
7. WhatsApp Channel
8. Deployment And Security
9. Workstream Ownership
10. Roadmap And Milestones
11. Commercial Pilot Plan
12. Risks And Controls
13. Review Checklist And Next Actions

Executive Summary

The Jhonny Retail Agent POC turns live Odoo data into a practical decision tool for a small retail owner. The product now has a local FastAPI backend, a Next.js web app, a richer analytics dashboard, an assisted chat surface, and a WhatsApp webhook scaffold that uses the same backend agent path.

The strongest product loop is simple: Jhonny opens the app, sees current sales, stock, purchases, and financial exposure, then asks the assistant what needs attention. The assistant answers from curated Odoo tools rather than free-form database access. This keeps the demo grounded, repeatable, and safer for a paid pilot.

The build remains local-first while the demo flow is stabilized. Cloud deployment should start only after review confirms the demo is strong enough and the team has a confirmed hosting account or subscription. A Render blueprint and Dockerfiles already exist as a reference path, while Azure Container Apps remains the preferred enterprise-aligned option once access is available.

1. Product Goal

Build a first POC that Jhonny can use and that can be shown to similar retailers as a paid-pilot offer. The product should not become a broad AI platform before the first commercial proof points. It should focus on daily owner decisions from systems the retailer already uses.

Capability	Current Description	Review Success Criteria
Live analytics app	Next.js app with home, analytics, and assisted agent surfaces	Reviewer can understand store performance without reading Odoo directly
Curated business agent	FastAPI <code>/chat</code> endpoint uses OpenAI first, Databricks fallback, or deterministic routing	Answers are grounded in tool output and include evidence metadata
Odoo intelligence tools	Read-only tools cover sales, stock, purchases, bills, receivables, margin, risk, and daily priorities	Tool catalogue matches the first buyer questions
WhatsApp channel	Webhook accepts Twilio form payloads and Meta WhatsApp JSON payloads	Approved sender can receive concise grounded answers
Repeatable pilot	Local-first workflow plus container deployment path	Next retailer can be onboarded without rewriting the product

2. Current Implementation

The current build is a working local POC:

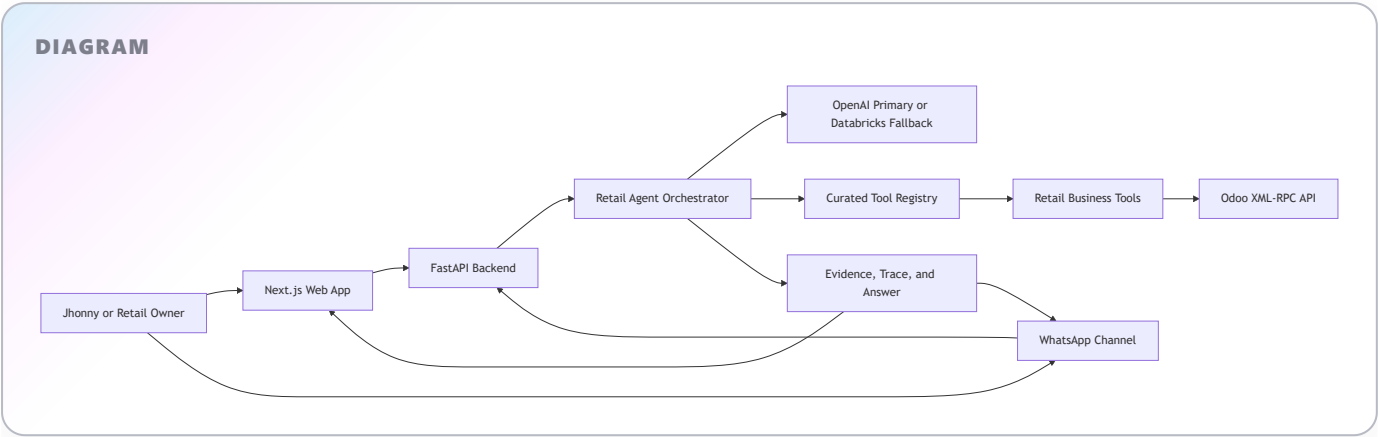
Component	Current State
Backend API	FastAPI app at <code>http://127.0.0.1:8000</code>
Web app	Next.js app at <code>http://127.0.0.1:3000</code>
Odoo integration	XML-RPC client with authentication, live reads, and retry handling for transient transport failures

Component	Current State
Agent	<code>RetailAgent</code> orchestrates curated tools, LLM selection, answer writing, evidence summaries, and fallback routing
Frontend shell	EY/EW-style branded app shell with theme support, profile menu, branded header, and client footer
Demo access	<code>APP_AUTH_TOKEN</code> protects app endpoints; frontend stores the demo token in local storage
Deployment assets	Root backend Dockerfile, <code>frontend/Dockerfile</code> , and <code>render.yaml</code> blueprint

Local development URLs:

Service	URL
Web app	<code>http://127.0.0.1:3000</code>
Backend API	<code>http://127.0.0.1:8000</code>
Health check	<code>http://127.0.0.1:8000/health</code>

3. Target Architecture



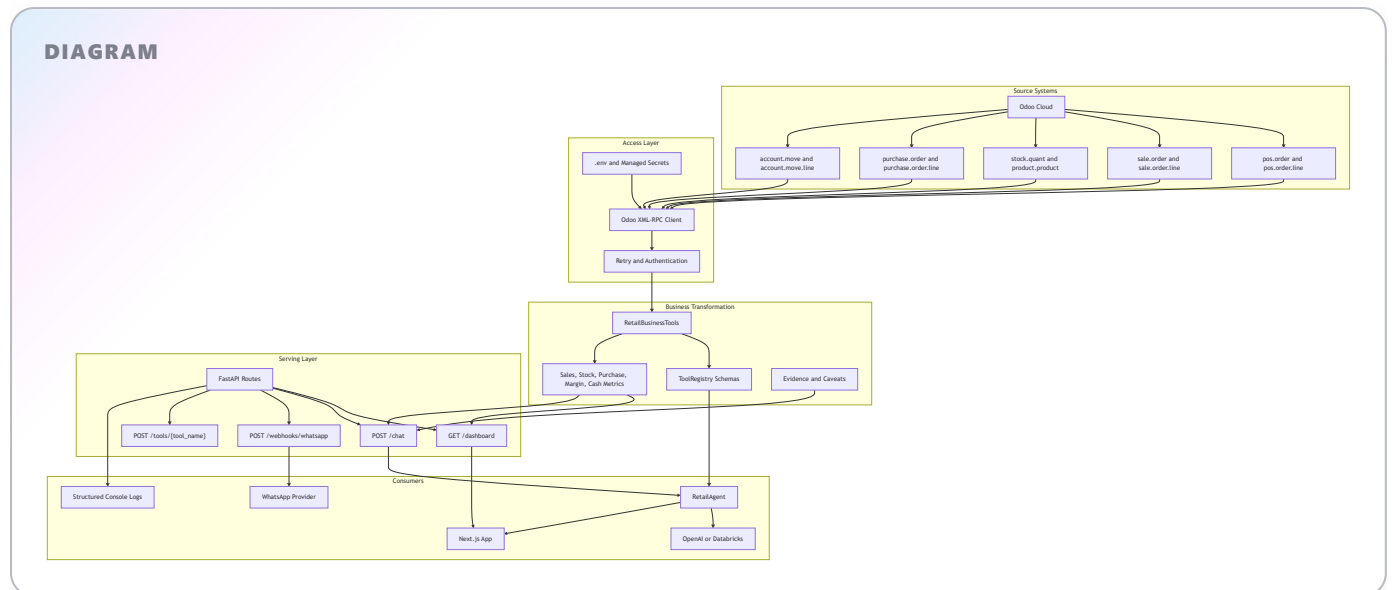
The app and WhatsApp channel share the same backend, agent, and tool registry. This matters because the team can improve one agent path and immediately benefit both channels.

The target product should preserve three constraints:

Constraint	Reason
Read-only operational access first	Reduces risk during pilot and avoids changing client records
Curated tools only	Prevents unsupported LLM claims and keeps answers explainable
Local-first until reviewed	Avoids cloud work before the demo and commercial story are validated

4. Data Architecture

The POC uses Odoo as the system of record. The backend reads live operational data through XML-RPC, transforms it into curated business views, and returns only the structured outputs needed by the app, agent, and WhatsApp channel. There is no separate warehouse, vector store, or committed business dataset in the current build.



Data Area	Source Models	Current Use
Sales	<code>pos.order</code> , <code>pos.order.line</code> , <code>sale.order</code> , <code>sale.order.line</code>	Today, month, YTD, daily trend, category, brand, product, weekday, and hour analysis
Stock	<code>stock.quant</code> , <code>product.product</code> , <code>product.template</code>	Stock value, stock age, available units, brand/category exposure, low stock, cover, and overstock risk
Purchases	<code>purchase.order</code> , <code>purchase.order.line</code>	Supplier spend, recent purchase orders, purchase-versus-sales signals, replenishment recommendations
Accounting exposure	<code>account.move</code> , <code>account.move.line</code>	Supplier bills, receivables, payables, working capital, and bill-line preview
Agent evidence	Tool results and trace summaries	User-facing answer grounding, request review, and demo explainability
Logs	FastAPI structured console output	Chat and WhatsApp events, request ID, selected tool, provider, latency, and success or failure

Data handling principles:

- Odoo remains the source of truth; the POC does not persist replicated business data.
- The backend exposes curated aggregates and selected operational details, not open-ended raw model access.
- LLM providers receive tool result JSON needed to answer the question, not direct Odoo credentials.
- WhatsApp responses should stay short and avoid customer personal data.
- Hosted pilots should move `.env` values into managed secrets and connect logs to the hosting provider.

5. App Experience

The app has three main surfaces: Home, Analytics, and Assisted Agent.

Surface	Current Content	Purpose
Home	Branded hero, store pulse, daily sales, daily profit, last week sales estimate, profit margin, YTD profit, YTD sales, stock value, supplier bill exposure	Show the value in the first 30 seconds
Analytics	Sales, Stock, Purchases, and Financials dashboards with period selector	Let reviewers inspect the operational data story
Assisted Agent	Chat interface, suggested prompts, session history, answer metadata, evidence trace, request ID	Let the owner ask business questions in plain language

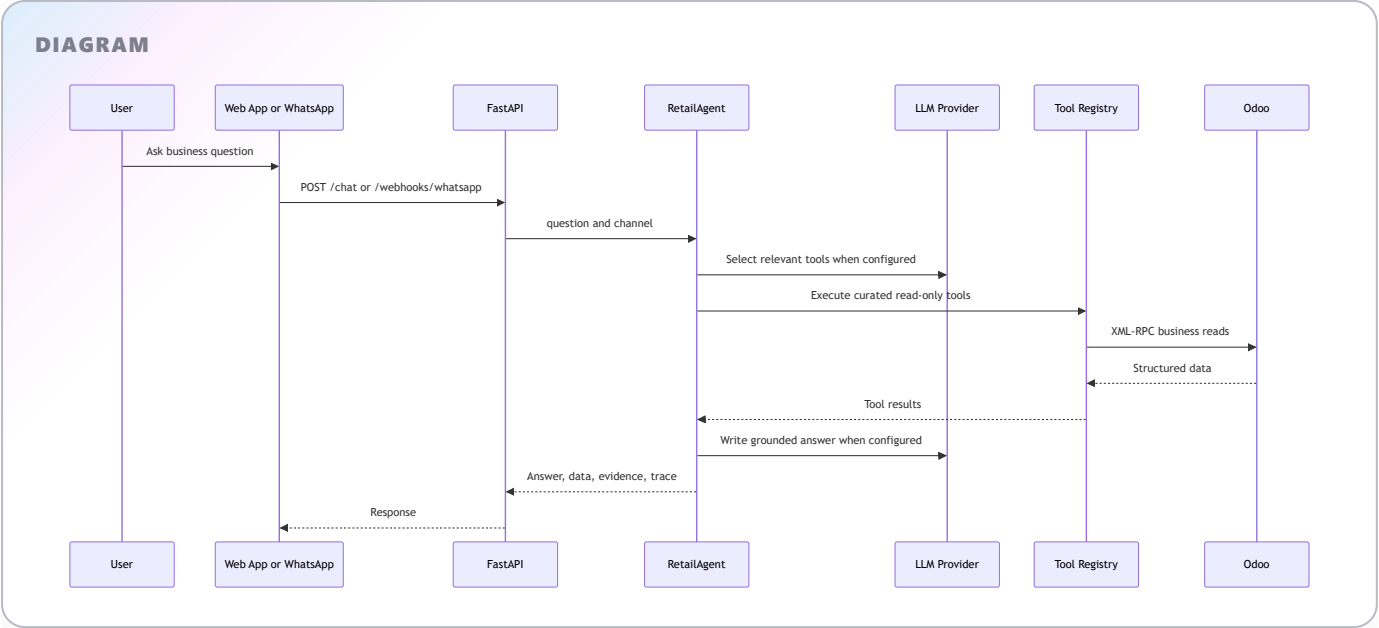
Current analytics dashboard coverage:

Dashboard	Included Views
Sales	Period sales, average daily sales, top product, margin, daily trend, weekday revenue, hourly sales, category ranking, brand ranking
Stock	Stock value, stock units, low-stock items, top stock brand, stock by brand, stock by category, stock antiquity, low-stock watchlist
Purchases	Period purchases, bills to pay, purchase ratio, supplier bill exposure, recent purchase orders, selectable open bill preview with line details
Financials	Receivables, payables, working capital, profitability view, cash exposure, period purchases versus sales

The app should continue to be optimized for a shop-owner demo rather than an internal BI analyst workflow. The interface should stay simple, visual, and action-oriented.

6. Agent And Tooling

The agent is grounded in a registry of read-only Odoo tools. With `OPENAI_API_KEY` configured, the agent uses OpenAI for tool planning and answer writing. If OpenAI is not configured, it attempts Databricks Model Serving if those environment variables are present. If no LLM provider is configured, deterministic routing keeps the local demo usable.



Current tool groups:

Group	Tools
Sales	<code>get_today_sales</code> , <code>get_daily_sales_series</code> , <code>get_month_sales</code> , <code>get_sales_by_category</code> , <code>get_sales_performance_breakdown</code> , <code>get_top_and_bottom_products</code> , <code>get_recent_orders</code>
Stock	<code>get_stock_value</code> , <code>get_stock_value_by_category</code> , <code>get_low_stock</code> , <code>get_stock_cover_and_velocity</code> , <code>get_dead_and_aged_stock</code>
Purchases	<code>get_purchase_summary</code> , <code>get_purchase_vs_sales_analysis</code> , <code>get_supplier_purchase_history</code> , <code>get_product_replenishment_insight</code>
Finance	<code>get_key_financials</code> , <code>get_open_bills</code> , <code>get_open_customer_invoices</code> , <code>get_profitability_snapshot</code> , <code>get_margin_by_product_category_brand</code> , <code>get_price_cost_exceptions</code> , <code>get_working_capital_snapshot</code> , <code>get_financial_risk_alerts</code>
Briefing	<code>get_business_snapshot</code> , <code>get_daily_owner_briefing</code> , <code>get_recommendation_for_question</code>

The answer payload can include:

Field	Purpose
<code>answer</code>	User-facing response
<code>tool</code>	Tool or tools used
<code>data</code>	Structured result for app inspection
<code>evidence</code>	Summary of supporting data
<code>tool_trace</code>	Tools, arguments, latency, and result summaries
<code>visualization</code>	Chart payload when a visual answer is appropriate
<code>llm_provider</code>	<code>openai</code> , <code>databricks</code> , <code>fallback</code> , or <code>not_configured</code>

Field	Purpose
request_id	Traceability for logs and review

7. WhatsApp Channel

The WhatsApp route is implemented as:

POST /webhooks/whatsapp

It accepts:

Provider Path	Payload	Response
Twilio WhatsApp sandbox or production	Form-encoded <code>From</code> and <code>Body</code>	Twiml XML response
Meta WhatsApp Cloud API	JSON webhook payload	JSON response with <code>answer</code> and <code>tool</code>

Production safeguards already represented in the backend:

Control	Current Behavior
Sender allowlist	<code>WHATSAPP_ALLOWED_NUMBERS</code> restricts approved phone numbers when set
Rate limiting	<code>WHATSAPP_RATE_LIMIT_PER_MINUTE</code> limits messages by sender
Twilio signature	<code>TWILIO_AUTH_TOKEN</code> validates <code>x-twilio-signature</code> against <code>PUBLIC_WHATSAPP_WEBHOOK_URL</code>
Meta signature	<code>WHATSAPP_APP_SECRET</code> validates <code>X-Hub-Signature-256</code>
Short channel answers	Agent keeps WhatsApp answers shorter than app answers
Sensitive data control	Agent prompt avoids customer personal data

WhatsApp should remain a secondary channel until the app demo is strong. It is valuable because it makes the owner experience immediate, but it should not drive scope creep before the core analytics and agent story are approved.

8. Deployment And Security

The current recommendation is still local-first until the review confirms demo readiness. When hosted access is available, deploy two services:

Service	Artifact	Notes
Backend API	Root <code>Dockerfile</code>	FastAPI, Odoo credentials, LLM credentials, WhatsApp secrets
Web app	<code>frontend/Dockerfile</code>	Next.js app with <code>NEXT_PUBLIC_API_BASE_URL</code>

Hosting options:

Option	Recommendation
Azure Container Apps	Preferred first enterprise path once Azure access exists
Render	Existing <code>render.yaml</code> blueprint is available for quick two-service deployment
AWS App Runner or ECS Fargate	Valid if AWS access arrives before Azure

Managed secrets needed for a hosted demo:

Area	Variables
Odoo	<code>ODOO_URL</code> , <code>ODOO_DB</code> , <code>ODOO_USERNAME</code> , <code>ODOO_API_KEY</code>
App auth and CORS	<code>APP_AUTH_TOKEN</code> , <code>APP_CORS_ORIGINS</code> , <code>NEXT_PUBLIC_API_BASE_URL</code>
WhatsApp	<code>WHATSAPP_ALLOWED_NUMBERS</code> , <code>WHATSAPP_RATE_LIMIT_PER_MINUTE</code> , <code>PUBLIC_WHATSAPP_WEBHOOK_URL</code> , <code>TWILIO_AUTH_TOKEN</code> , <code>WHATSAPP_APP_SECRET</code>
LLM	<code>OPENAI_API_KEY</code> , <code>OPENAI_MODEL</code> , optional <code>DATABRICKS_HOST</code> , <code>DATABRICKS_TOKEN</code> , <code>DATABRICKS_MODEL_ENDPOINT</code>

Security review notes:

- Rotate any Odoo API key used during development before production or paid pilot hosting.
- Keep `.env` local and out of commits.
- Keep app endpoints protected with `APP_AUTH_TOKEN` until a proper login model is needed.
- Do not expose customer personal data in WhatsApp responses.
- Treat estimated profitability and margins as decision support, not statutory accounting output.

9. Workstream Ownership

Owner	Primary Focus	Current Deliverables
Tiago	Web app and analytics experience	Home, Analytics, Assisted Agent, frontend polish, mobile-friendly demo flow
Rodrigo	WhatsApp feature	Provider decision, webhook setup, phone allowlist, message testing, signature configuration
Shared	Backend, Odoo tools, LLM, deployment, validation	FastAPI API, tool registry, OpenAI/Databricks routing, Docker path, smoke tests
Founding team	Commercial pilot readiness	Demo script, pricing, outreach, pilot qualification, client feedback loop

10. Roadmap And Milestones

Phase	Timing	Goal	Owner	Exit Criteria
1	Now	Review-ready local app and plan	Shared	App opens locally, dashboard loads, chat answers from Odoo tools
2	Next	Demo polish and validation	Tiago	5-minute demo works without engineering explanation
3	Next	WhatsApp provider test	Rodrigo	Approved number receives correct answer through webhook
4	Next	Hosted review environment	Shared	Managed secrets, HTTPS, CORS, auth token, health checks, logs
5	After Jhonny feedback	Paid pilot outreach	Founding team	10 to 20 qualified retailers contacted with clear pilot offer
6	After 3 to 5 pilots	Productization decision	Founding team	Repeatable pain, pricing, onboarding effort, and usage evidence are proven

The next milestone should be a stable owner demo where the reviewer can see live data, ask a question, inspect the evidence, and understand the paid-pilot offer.

11. Commercial Pilot Plan

The first paid offer should stay narrow and outcome-focused.

Item	Recommendation
Setup fee	EUR 1,500 to EUR 3,000
Monthly pilot fee	EUR 300 to EUR 900
Anchor offer	EUR 2,000 setup plus EUR 500 per month
Pilot duration	2 to 4 weeks
Target clients	Small retailers with 1 to 10 stores
First verticals	Surf, outdoor, fashion, footwear, sports equipment, lifestyle retail

Do not sell the product as generic AI. Sell it as fast daily business answers from systems the retailer already uses.

Pilot scope:

Included	Excluded For First Pilot
One operational system connection, starting with Odoo	Forecasting engine
Core dashboards for sales, stock, purchases, and financial exposure	Custom ERP workflows

Included	Excluded For First Pilot
Assisted questions in the web app	Deep accounting reconciliation
WhatsApp owner questions when provider access is available	Full multi-tenant SaaS admin
Weekly review call and end-of-pilot recommendation	Custom data science project

12. Risks And Controls

Risk	Impact	Control
Metrics do not match Odoo UI	Loss of trust	Validate core numbers with Jhonny before external demos
Estimated margin is mistaken for accounting profit	Bad decisions	Label margin as estimated from standard cost and include caveats
WhatsApp exposes sensitive data	Security risk	Restrict numbers, verify signatures, avoid customer personal data
LLM invents unsupported claims	Trust risk	Force answers through curated tools and evidence summaries
Development credentials leak	Security risk	Keep <code>.env</code> ignored, rotate Odoo key before hosted demo
App becomes too broad before selling	Slow revenue	Keep pilot scope around sales, stock, purchases, margin, and daily priorities
Cloud work starts too early	Delivery risk	Host only after demo review and access are confirmed
Odoo field quality varies by client	Onboarding risk	Document required models and add validation checks per new retailer

13. Review Checklist And Next Actions

Review checklist:

Check	Expected Result
Backend health	<code>GET /health</code> returns <code>{"status":"ok"}</code>
Dashboard	<code>GET /dashboard</code> returns live Odoo metrics when <code>X-App-Token</code> is valid
Agent chat	<code>POST /chat</code> returns answer, tool, evidence, trace, provider, and request ID
OpenAI path	<code>scripts/smoke_openai_chat.py</code> confirms <code>llm_provider</code> is <code>openai</code> when credentials are set
Security checks	<code>scripts/evaluate_api_security.py</code> passes sender allowlist, rate limit, and signature tests
Agent routing	<code>scripts/evaluate_agent.py</code> passes deterministic routing checks
Frontend build	<code>npm run build</code> passes in <code>frontend/</code> before hosted deployment

Immediate next actions:

Priority	Action	Owner
1	Run the app locally and rehearse the 5-minute review demo	Tiago
2	Validate sales, stock, purchase, and bill numbers against Odoo UI	Shared
3	Run OpenAI smoke test before any external review	Shared
4	Decide whether the review needs WhatsApp live or app-only demo	Rodrigo and founding team
5	If WhatsApp is included, configure provider, public URL, allowlist, and signature secret	Rodrigo
6	Prepare hosted environment only after local demo approval	Shared
7	Use Jhonny feedback to qualify 10 to 20 similar retailers for paid pilots	Founding team

The POC is now strong enough to review as a focused retail intelligence demo. The team should use the review to decide whether to polish for Jhonny first, connect WhatsApp live, or move directly into a hosted pilot environment.